Appendix B

Maintenance Manual

Project: Car License Plate Recognition Smart Parking System
Student Name: Li Wenhao **Student ID:** 50087574

This Maintenance Manual provides full technical details for developers, maintainers, and administrators to install, build, modify, extend, debug, and deploy the system. All implementation structures, dependencies, file organisation, constants, core logic, and known limitations are fully documented to ensure long-term maintainability and extensibility, and are 100% aligned with the actual project codebase.

B.1 System Overview

This project is a full-stack web-based Automatic License Plate Recognition (ALPR) system with cross-regional car license plate recognition capability, integrated with end-to-end smart parking management. It adopts a standardised front-end and back-end separation architecture, with a dedicated AI processing pipeline for high-accuracy license plate detection and character recognition.

B.1.1 Core System Functional Scope

The system is designed to automate the full workflow of vehicle parking management, covering vehicle entry registration, license plate recognition, parking duration tracking, fee calculation, exit verification, and historical record management, with role-based user authentication for system security.

B.1.2 Core Functional Modules

- Automatic car license plate detection and character recognition
- Vehicle entry & exit event registration and management
- Real-time parking duration calculation and parking fee auto-calculation
- JWT-based role user authentication and access control
- Parking record query, management, and system configuration functions

B.1.3 AI Processing Pipeline

The end-to-end recognition workflow follows a standardised pipeline:

Image Input $\rightarrow$ *Image Pre-processing & Enhancement* $\rightarrow$ *License Plate Region Detection* $\rightarrow$ *Perspective Correction* $\rightarrow$ *Optical Character Recognition (OCR)* $\rightarrow$ *Result Output*

B.1.4 Core Detection Model Source (`best.pt`)

The model file `best.pt` is a custom fine-tuned YOLOv8s model for car license plate detection. Its complete training configuration and origin are documented as follows:

1. **Base Model**: YOLOv8s, the small-scale efficient object detection model provided by Ultralytics.

2. **Training Dataset**: The public *Car License Plate Detection* dataset published on Kaggle (`https://www.kaggle.com/datasets/andrewmvd/car-plate-detection`), which contains 433 labelled images with license plate bounding box annotations in PASCAL VOC format.

3. **Training Platform**: AutoDL Cloud Computing Platform (high-performance GPU rental for deep learning training).

4. **Training Environment & Image**: Docker mirror `ultralytics/ultralytics:yolov8_firedataset:v1` (pre-configured Ultralytics YOLO environment).

5. **Training Hardware**: GPU: NVIDIA RTX 5090 (32 GB) $\times$ 1; CPU: 25 vCPU Intel(R) Xeon(R) Platinum 8470Q.

6. **Training Purpose**: Fine-tuned to improve the accuracy and stability of license plate detection under real-world parking scenarios, including tilted angles, partial occlusion, and varying lighting conditions.

7. **Fixed File Path**: `backend/weights/best.pt`

8. **System Role**: Detects and locates the license plate region from input vehicle images, providing a standardised cropped image for subsequent perspective correction and OCR processing.

B.2 System Architecture

The system adopts a standard layered front-end and back-end separation architecture, with clear module boundaries to ensure maintainability and independent iteration of each component.

B.2.1 Core Architecture Layers

Presentation Layer (Frontend) Vue3 + Vite, responsible for user interface rendering, user interaction, and data visualisation.

Application Layer (Backend) Python + FastAPI, responsible for API routing, business logic processing, authentication, and database interaction.

AI Processing Layer YOLOv8s detection + fast-plate-ocr + image enhancement + perspective correction, responsible for the core license plate recognition workflow.

Data Persistence Layer SQLite file-based database, responsible for persistent storage of parking records, user information, and system configuration.

B.2.2 End-to-End Data Flow

User Image Input → Frontend Request → Backend API Routing → AI Processing Pipeline → Business Logic Processing → Database Operation → API Response → Frontend Interface Rendering

B.3 Hardware and Software Dependencies

B.3.1 Minimum Hardware Requirements

- CPU: Multi-core Intel/AMD 2.0 GHz or above (tested on 12th Gen Intel Core i7-12700H)
- RAM: Minimum 8 GB (16 GB recommended; tested on 16 GB)
- Storage: Minimum 10 GB free disk space
- Network: Required for dependency installation and package download

B.3.2 Software Environment Requirements

- Python: 3.10–3.11 (stable release, 64-bit)
- Node.js: 16.x–18.x (LTS version)
- Operating System: Windows 10/11 64-bit (compatible with Linux/macOS with minor path adjustments)
- Browser: Chrome or Firefox (latest 2 versions recommended)

B.3.3 Core Dependent Libraries

- **Backend Core Libraries**: `fastapi`, `uvicorn`, `ultralytics`, `fast-plate-ocr`, `opencv-python`, `sqlalchemy`, `python-jose`, `passlib`, `python-multipart`
- **Frontend Core Libraries**: `vue3`, `vite`, `axios`, `element-plus`, `vue-router`

B.4 Installation Instructions

B.4.1 Environment Preparation

1. Install Python 3.10–3.11, and add Python and its Scripts directory to the system PATH.
2. Install Node.js 16.x–18.x LTS, and add Node.js and npm to the system PATH.
3. Verify the installation with the following commands in the terminal:

```
python --version
node --version
npm --version
```

B.4.2 System Entry Point

The core system entry point for backend service startup is `backend/app/main.py`.

B.4.3 One-Click Installation and Startup

For first-time use, double-click the `first_start.bat` script in the project root directory. The script will automatically complete dependency installation for both front-end and back-end, initialise the database, and start the system service.

B.4.4 Step-by-Step Manual Installation

Backend Dependency Installation:

```
cd backend
pip install -r requirements.txt
```

Frontend Dependency Installation:

```
cd frontend
npm install
```

B.5 Compilation and Build Instructions

This system uses interpreted languages for both front-end and back-end, and does not require low-level compilation.

B.5.1 Development Mode Startup

Backend Service Startup:

```
cd backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend Development Server Startup:

```
cd frontend
npm run dev
```

B.5.2 Production Environment Build

Frontend Production Build:

```
cd frontend
npm run build
```

Production build output directory: `frontend/dist`

Note: A warning about chunk size exceeding 500 kB may appear during build, which is a normal development environment prompt and does not affect runtime functionality or system stability.

B.6 System File Organisation

The full project file structure is 100% aligned with the actual codebase, as follows:

```
root/
|-- backend/                # Backend service & AI core modules
|   |-- app/
|   |   |-- algorithm/      # AI recognition core algorithm modules
|   |   |   |-- pipeline.py  # End-to-end LPR pipeline entry
```

```
|   |   |   |-- correction/  # Perspective correction module
|   |   |   |-- detection/   # YOLOv8 detection module
|   |   |   |-- enhancement/ # Image enhancement module
|   |   |   `-- recognition/ # OCR recognition module
|   |   |-- api/             # API endpoint definitions & routing
|   |   |-- services/        # Core business logic implementation
|   |   |-- models/          # Database ORM models & connection
|   |   |-- core/            # System configuration & security
|   |   |-- schemas/         # Data validation schemas
|   |   |-- utils/           # General helper functions & tools
|   |   `-- main.py          # Backend service entry point
|   |-- weights/             # Trained YOLOv8 model file storage
|   |   `-- best.pt
|   |-- logs/                # System runtime log files
|   |-- uploads/             # Intermediate image storage
|   |-- .env                 # Environment variable configuration
|   |-- Dockerfile           # Containerization configuration
|   |-- parking.db           # SQLite database (auto-generated)
|   |-- pricing_config.json  # Pricing config (auto-generated on first save)
|   `-- requirements.txt     # Backend dependency list
|-- frontend/                # Frontend web interface project
|   |-- src/
|   |   |-- views/           # Page components
|   |   |-- router/          # Frontend routing configuration
|   |   |-- api/             # Frontend API request encapsulation
|   |   |-- components/      # Reusable UI components
|   |   |-- assets/          # Static resource files
|   |   `-- main.js          # Frontend project entry point
|   |-- package.json         # Frontend dependency list
|   |-- vite.config.js       # Vite build configuration
|   `-- .env.development     # Frontend environment configuration
|-- test_images/             # Standard test images for validation
|-- first_start.bat          # One-click installation & startup script
|-- start.bat                # Quick startup script
|-- project_structure.txt    # Project structure documentation
`-- README.md                # Project overview & quick start guide
```

B.6.1 Key File Fixed Locations

- Trained AI Detection Model: `backend/weights/best.pt`
- SQLite Database File: `backend/parking.db` (auto-generated on first backend startup)
- Database ORM Models & Connection: `backend/app/models/`
- Core AI Algorithm Modules: `backend/app/algorithm/`
- API Route Definitions: `backend/app/api/v1/endpoints/`
- System Security Configuration: `backend/app/core/security.py`

- Parking Business Logic: `backend/app/services/parking_service.py`

B.7 Space and Memory Requirements

- Total Project Disk Space Occupancy: $\sim$10 GB (including dependencies, model weights, and runtime cache)
- Runtime RAM Occupancy: 2 GB–4 GB under normal operation
- Core AI Model File Size: $\sim$30 MB
- Initial Database File Size: $<$100 KB (grows with parking record volume)

B.8 Source Code Files and Core Roles

B.8.1 Backend Core Files and Functions

`main.py` Backend service entry point, responsible for application initialisation, database table creation, middleware registration, and route mounting.

`pipeline.py` End-to-end LPR pipeline entry, coordinates image enhancement, detection, perspective correction, and OCR.

`enhancer.py` Low-light image enhancement module, performs bilateral filtering, CLAHE contrast enhancement, and HSV adaptive brightness adjustment.

`detector.py` YOLOv8s license plate detection module, loads the fine-tuned model and detects plate regions.

`perspective.py` Perspective correction module, corrects tilted license plates via 4-point contour detection and `warpPerspective` transformation.

`ocr_engine.py` OCR character recognition module, uses fast-plate-ocr to extract text from corrected license plate images.

`security.py` JWT authentication and password encryption module, handles user identity verification and access control.

`database.py` Database connection and session management module, provides database engine and session factory.

`parking_service.py` Core parking business logic module, handles vehicle entry/exit, parking duration calculation, and fee calculation. Reads pricing parameters dynamically from `pricing_config.json` (falling back to `config.py` defaults) to ensure fee calculation always uses the latest configured rate.

`pricing.py` Pricing configuration API endpoint, provides `GET /pricing/config` and `PUT /pricing/config` for querying and updating the free parking hours and hourly rate, persisting changes to `pricing_config.json`.

B.8.2 Frontend Core Files and Functions

`main.js` Frontend project entry point, initialises Vue application, registers components, and configures global settings.

`Login.vue` User login page, handles identity verification and JWT token storage.

`Entry.vue` / `Exit.vue` Parking entry/exit pages, handle image upload, recognition, and record creation.

`List.vue` Parking record management page, handles historical record query, filtering, and display.

`vite.config.js` Frontend build and development server configuration file.

B.9 Crucial Constants and Code Locations

All configurable system constants and their fixed code locations are documented below for easy modification and maintenance:

- Backend Service Port: 8000 → defined in `backend/main.py` and startup command
- Frontend Development Server Port: 3001 → defined in `frontend/vite.config.js`
- JWT Secret Key & Expiration Time → defined in `backend/app/core/config.py`
- Image Brightness Threshold for Enhancement → defined in `backend/app/core/config.py` (`BRIGHTNESS_LOW_THRESHOLD`, `BRIGHTNESS_HIGH_THRESHOLD`)
- Fixed Model File Path → defined in `backend/app/algorithm/detection/detector.py`
- Parking Free Hours & Hourly Rate → defined in `backend/app/core/config.py` (default values); dynamically overridden by `backend/pricing_config.json` when updated via the Price Management API
- Database File Path → defined in `backend/app/core/config.py` (points to `backend/parking.db`)

B.10 Main Classes and Methods

The core classes and methods are 100% aligned with the actual codebase, documented below:

B.10.1 Core AI Algorithm Classes

LRPPipeline (`app/algorithm/pipeline.py`) `run_recognition()`: End-to-end license plate recognition workflow entry, coordinates image enhancement, plate detection, perspective correction, OCR, and returns final results.

ImageEnhancer (`app/algorithm/enhancement/enhancer.py`) `enhance_image()`: Performs bilateral filtering, CLAHE contrast enhancement, and HSV adaptive brightness adjustment on input images.

PlateDetector (`app/algorithm/detection/detector.py`) `detect_plate()`: Loads the fine-tuned YOLOv8s model, detects license plate regions, returns bounding box coordinates and cropped plate images.

PerspectiveCorrector (`app/algorithm/correction/perspective.py`) `correct_perspective()`: Corrects perspective distortion of tilted license plates via 4-point contour detection and `cv2.warpPerspective` transformation.

OCREngine (`app/algorithm/recognition/ocr_engine.py`) `recognize_characters()`: Uses fast-plate-ocr to extract character content from corrected license plate images.

B.10.2 Core Business Classes

ParkingService (`app/services/parking_service.py`) `create_parking_record()`: Handles vehicle entry logic, verifies no active record exists, creates a new parking record in the database.
`process_vehicle_exit()`: Handles vehicle exit logic, matches active parking records, calculates parking duration and fees, updates record status.

B.10.3 Database Core Functions

get_db() (`app/models/database.py`) Provides thread-safe database sessions for all business logic CRUD operations.

Database Initialisation Executed in `main.py` via `Base.metadata.create_all(bind=engine)`, creates all required database tables on first startup.

B.11 Critical File Path Reference

For maintenance and modification, the following fixed file paths are referenced:

- Core AI Detection Model: `backend/weights/best.pt`
- Backend Dependency Configuration: `backend/requirements.txt`
- Frontend Dependency Configuration: `frontend/package.json`
- SQLite Database File: `backend/parking.db`
- Test Image Set: `test_images/`
- System Entry Point: `backend/app/main.py`
- System Configuration: `backend/app/core/config.py`

B.12 Known Issues and Limitations

This section documents the confirmed known issues and inherent limitations of the current system version, to guide maintenance and subsequent optimisation:

1. License plates with a tilt angle greater than 45° will have significantly reduced detection and recognition accuracy, due to the limited coverage of tilted samples in the training dataset.
2. Extreme low-light environments (illumination <5 lux) cannot be fully corrected by the current image enhancement algorithm, which may lead to recognition failure.

3. The default ports 8000 (backend) and 3001 (frontend) may be occupied by other applications, resulting in service startup failure.

4. npm will report development dependency vulnerability warnings during frontend installation, which are limited to the build environment and do not pose any runtime security risks.

5. The system's format validation rules are configured for a specific regional license plate format by default. Adapting to other regional formats requires updating the business logic validation rules in the service layer.

B.13 Future Improvements and Extensibility

The following optimisations and functional extensions are planned for subsequent iterations, to improve system performance and expand application scenarios:

1. Extend GPU acceleration support for the AI processing pipeline beyond the current configurable device setting, including multi-GPU inference and dynamic batching, to further reduce recognition latency under high-throughput scenarios.

2. Optimise the perspective correction algorithm, to improve detection and recognition accuracy for license plates with large tilt angles.

3. Upgrade the low-light image enhancement module with a dedicated deep learning model, to adapt to extreme lighting conditions.

4. Add support for real-time camera stream recognition, to achieve fully automated unmanned parking management.

5. Add parking record export function, supporting Excel and PDF format export for management and archiving.

6. Optimise the system for cloud server deployment, supporting containerised deployment via Docker.

7. Add multi-role permission management, to adapt to the management needs of different parking lot staff.

B.14 Debugging and Troubleshooting

This section provides standard troubleshooting solutions for common faults during system operation and maintenance:

1. **Dependency installation failed**: Check network connectivity, confirm Python and Node.js versions are within the required range, and use a domestic mirror source if necessary.

2. **Backend service cannot start**: Check if port 8000 is occupied by other applications, confirm all dependencies are installed successfully, and verify the model file exists in `backend/weights/best.pt`.

3. **Frontend cannot connect to the backend**: Confirm the backend service is running normally, check the backend address configuration in the frontend API encapsulation, and ensure no firewall blocks the port.

4. **Recognition function unavailable**: Check if the input image format is supported (JPG/PNG recommended), verify the model file path is correct, and confirm all AI-related dependencies are installed completely.

5. **Database operation failed**: Check the write permission of the `backend` directory, confirm the `parking.db` file is not locked by other processes, and restart the service to reinitialise the database if necessary.

6. **Perspective correction not working**: Ensure the detected plate region is a clear quadrilateral; extreme distortion or occlusion may prevent contour detection.